

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

Duration : 1

Hour

QUESTIONS: 5

Total

Marks

Reverse Engineering

Code of Conduct:

I BAIRAGANI DINESH-192372189 (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:BAIRAGANI DINESH

Annexure A

Reverse Engineering

1. Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

1. Problem Identification

Reverse engineering involves understanding the structure and behavior of software when its source code or documentation is unavailable. Manually analyzing large numbers of programs is time-consuming. Machine Learning can automate code classification, similarity detection, and anomaly identification.

2. Objective

- Classify software components automatically.
- Identify similar programming patterns.
- Detect unusual code.
- Predict the functionality of unknown code.
- Compare different ML algorithms.

3. Methodology

Source Code Dataset → Feature Extraction → Data Preprocessing → ML Model → Classification/Clustering → Evaluation

A dataset can contain safe source-code samples from different programs.

Features

- Number of lines of code
- Number of functions
- Number of classes
- Keywords
- Operators
- API/function-call patterns
- Cyclomatic complexity
- Token frequency

4. ML Algorithms

Supervised Learning

- Random Forest
- Decision Tree
- SVM

Used when source-code samples have known labels.

Unsupervised Learning

- K-Means
- DBSCAN

Used to group similar programs without predefined labels.

Semi-Supervised Learning

Uses a small amount of labelled code together with a large amount of unlabelled code.

5. Example Python Workflow

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier(n_estimators=100)
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, prediction))
```

6. Evaluation

Algorithms can be compared using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

7. Tools

- Python
- Pandas
- Scikit-learn
- Jupyter Notebook
- VS Code
- Radare2/Ghidra for safe software analysis

8. Expected Result

The ML system identifies similarities and differences between programs and classifies unknown code based on learned patterns.

Conclusion

Machine Learning makes reverse engineering faster and more scalable by automatically extracting code features and identifying software patterns. Different algorithms can be compared to determine the most suitable approach.

2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

1. Problem Identification

Large software repositories may contain thousands of files, functions, classes, and dependencies. Understanding an unfamiliar project manually requires significant time. AI can automate source-code analysis and generate human-readable explanations.

2. Objective

- Identify functions and classes.
- Understand variables and dependencies.
- Detect programming patterns.
- Generate natural-language explanations.
- Verify AI-generated explanations against actual code.

3. Methodology

Source Repository → Code Parsing → AI Analysis → Function/Class Identification → Dependency Analysis → Explanation → Human Verification

4. AI Analysis

The AI system analyzes:

- Functions

- Classes
- Variables
- Imports
- API calls
- Control flow
- Dependencies
- Comments and documentation

For example, if a function processes user authentication, the AI can explain its input, processing steps, output, and dependencies in simple language.

5. Tools

- Python
- AST parser
- GitHub
- VS Code
- Code analysis tools
- Large Language Models

6. Example

```
def calculate_total(price, quantity):  
    return price * quantity
```

AI-generated explanation:

“The function accepts price and quantity as inputs and returns their product, representing the total cost.”

The explanation must then be compared with the actual implementation.

7. Verification

The generated explanation is checked for:

- Correct function purpose
- Correct input/output
- Correct dependencies
- Missing conditions

- Incorrect assumptions

8. Advantages

- Faster code understanding.
- Useful for legacy software.
- Reduces documentation effort.
- Helps developers understand unfamiliar projects.

9. Limitations

- AI may misunderstand complex logic.
- Generated explanations can contain incorrect assumptions.
- Human verification is required.
- Proprietary code should not be unnecessarily exposed to external AI systems.

Conclusion

AI can significantly improve program understanding by converting complex source code into structured and human-readable information. However, generated results must always be verified against the actual code.

3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples.**

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware analysis.

1. Problem Identification

Malware analysis requires identifying suspicious programs and understanding their behavior. Manually analyzing large malware datasets is difficult. AI can help security researchers classify samples using safe, non-executable features.

2. Objective

- Identify suspicious software patterns.
- Classify samples as benign or malicious.
- Analyze static features.
- Detect unusual API and code patterns.
- Improve automated malware analysis.

3. Safe Analysis Method

Research Dataset → Static Feature Extraction → Preprocessing → ML Classification → Risk Prediction → Security Analysis

The analysis should use **non-executable datasets or isolated research samples**.

4. Static Features

Examples include:

- File metadata
- Strings
- Imported functions
- API-call indicators
- Section information
- Byte-level statistical features
- Entropy
- Control-flow characteristics

5. Machine Learning

Possible algorithms:

- Random Forest
- Decision Tree
- SVM

- Logistic Regression

Example:

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(n_estimators=100)
```

```
model.fit(X_train, y_train)
```

```
result = model.predict(X_test)
```

6. Evaluation

Performance can be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

For security applications, **precision and recall** are particularly important because false positives and false negatives can have significant consequences.

7. Intelligent Analysis

The system can identify samples with unusual feature combinations and assign a risk classification for further human investigation.

8. Tools

- Python
- Pandas
- Scikit-learn
- Jupyter Notebook
- Ghidra/Radare2 for controlled research analysis

9. Ethical and Safety Considerations

Malware should not be executed on normal computers. Analysis should use safe datasets, isolated environments, and authorized research samples.

Conclusion

AI can assist malware reverse engineering by automatically analyzing static features and classifying suspicious samples. It reduces manual workload while allowing security researchers to focus on detailed investigation.

4. Reverse Engineering of APIs Using AI

Description:

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.
- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery.

1. Problem Identification

When API documentation is incomplete, developers may need to understand the API by examining its requests, responses, source code, and observed behavior. AI can help recover API documentation and relationships between endpoints.

2. Objective

- Identify API endpoints.
- Understand HTTP methods.
- Identify parameters.
- Analyze request and response formats.
- Generate documentation.

- Compare generated documentation with the actual specification.

3. Methodology

Controlled API → Request/Response Collection → AI Analysis → Endpoint Identification → Parameter Analysis → Documentation Generation → Verification

4. API Information

The system analyzes:

- GET, POST, PUT, DELETE methods
- Endpoint paths
- Query parameters
- Request body
- Response body
- HTTP status codes
- Authentication requirements
- Data relationships

Example:

GET /api/products/101

Possible interpretation:

Purpose: Retrieve product information.

Input: Product ID = 101

Output: Product details in JSON format.

5. AI-Based Processing

AI compares multiple request-response examples to infer:

- Endpoint purpose
- Required parameters
- Optional parameters
- Data types
- Response fields
- Error conditions

6. Tools

- Python
- Postman
- Swagger/OpenAPI
- Browser Developer Tools
- Requests library
- LLM-based code analysis

7. Verification

Generated documentation is compared with the actual API specification.

Feature	AI Documentation	Actual API
Endpoint	/api/products/{id}	/api/products/{id}
Method	GET	GET
Parameter	Product ID	Product ID
Response	JSON	JSON

8. Advantages

- Reduces documentation effort.
- Helps understand legacy APIs.
- Identifies missing documentation.
- Quickly summarizes API behavior.

9. Limitations

- AI may infer incorrect behavior.
- Some endpoints may require authentication.
- Observed behavior may not represent every possible case.
- Verification is necessary.

Conclusion

AI-assisted API reverse engineering can recover useful documentation from controlled observations and source code. Combining AI analysis with human verification provides

a faster and more reliable approach to understanding undocumented or poorly documented APIs.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and	Problem clearly defined with a logical	Problem identified but analysis or	Problem statement unclear or incorrect; no

		advanced methods	engineering approach	methodology is weak	logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness